

SAAS API SECURITY ASSESSMENT

Postman-Based Security Testing Report

Prepared by: Gaddam Snehal

Cyber Security | API Security Assessment

Testing Tool: Postman

Test API: JSONPlaceholder (public demo/testing API)

Assessment Type: Authorized educational testing

Evidence Count: 10 screenshots

1. Executive Summary

This report documents a structured API security assessment performed using Postman against the JSONPlaceholder public demo API. The objective was to practice common SaaS/API security testing techniques including authentication review, object-level access checks, response-data review, HTTP status-code validation, security-header inspection, rate-limit observation, and POST request testing.

The assessment identified several security-relevant observations that would be important in a real production SaaS API. However, JSONPlaceholder is intentionally a public testing service, so the observations below are not presented as confirmed vulnerabilities in a production system. Instead, they demonstrate what a tester should verify in a real application and how the corresponding controls could be implemented.

2. Scope and Test Environment

Application/API	JSONPlaceholder public demo API
Base URL	https://jsonplaceholder.typicode.com
Testing Tool	Postman
Primary Endpoints Tested	GET /users, GET /users/1, GET /users/2, GET /users/99999, GET /posts/1, POST /posts
Authentication Observed	No Auth
Evidence	10 Postman screenshots
Assessment Nature	Educational / authorized testing of a public demo API

3. Testing Methodology

- Sent baseline GET requests and verified HTTP response status and JSON response structure.
- Reviewed the Authorization tab to determine whether authentication was configured.
- Compared object-specific requests such as /users/1 and /users/2.
- Tested an invalid object identifier (/users/99999) to verify error handling.
- Inspected response headers for security-related controls and rate-limit information.
- Reviewed returned JSON for potentially excessive user information.
- Tested a POST request to verify creation behavior and HTTP 201 Created handling.
- Captured screenshots as evidence for each significant test result.

4. Test Results Summary

Test	Endpoint / Area	Observed Result	Assessment
Authentication review	GET /users/1	No Auth; 200 OK	Security design observation
User data exposure review	GET /users/1	Multiple user fields returned	Review minimization in production
Object access comparison	GET /users/1 and /users/2	Both return 200 OK	Not proof of IDOR on demo API
Invalid object handling	GET /users/99999	404 Not Found	Expected behavior
Header review	GET responses	Security/rate-limit headers observed	Positive control indicators
POST behavior	POST /posts	201 Created	Expected behavior

5. Security Observations and Recommendations

5.1 Authentication — No Auth Observed

The tested requests were configured with 'No Auth' in Postman and still received successful responses. For a public demo API this is expected. In a real SaaS application, endpoints containing private customer or account information should normally require authentication.

Potential production risk: unauthorized users could access protected resources if authentication is missing.

Recommended controls:

- Require authentication on protected endpoints.
- Use secure token-based authentication such as OAuth 2.0/OpenID Connect where appropriate.
- Validate token expiry, issuer, audience, and signature.
- Return appropriate 401/403 responses when authentication or authorization fails.

5.2 Excessive Data Exposure — Response Contains Multiple User Fields

The user response contains several fields such as name, username, email, address, phone and other profile-related information. JSONPlaceholder is a mock API, so this is not treated as a confirmed vulnerability. In a production SaaS API, responses should expose only the fields required by the client and the user's authorization level.

Recommended controls:

- Define response schemas for every endpoint.
- Return only data required for the business function.
- Apply field-level authorization to sensitive attributes.
- Avoid returning internal or unnecessary personal information.

5.3 Object-Level Authorization / IDOR Check

The assessment compared object-specific requests for /users/1 and /users/2 and confirmed that different objects could be requested without authentication. This demonstrates the testing technique used to identify Broken Object Level Authorization (BOLA/IDOR). Because JSONPlaceholder is a public demo API and does not represent private user accounts, the evidence does not establish a production IDOR vulnerability.

Recommended production verification:

- Authenticate as User A and request User B's object identifier.
- Confirm that the server checks ownership/tenant authorization server-side.
- Return 403 Forbidden (or an appropriate non-disclosing response) when access is not permitted.
- Use authorization middleware consistently across every object-level endpoint.

5.4 Invalid Object Handling

A request to /users/99999 returned HTTP 404 Not Found. This indicates that the API handles a nonexistent object with a standard not-found response. Consistent error handling should be maintained in production without revealing unnecessary internal implementation details.

5.5 Security Headers and Rate Limiting

The captured response headers include controls such as X-Content-Type-Options: nosniff and rate-limit-related headers including X-Ratelimit-Limit and X-Ratelimit-Remaining. These are useful indicators of defensive controls.

In a production SaaS API, rate limiting should be applied according to endpoint sensitivity, user identity, IP reputation, and business requirements, with appropriate monitoring and alerting.

5.6 POST Request and Status-Code Validation

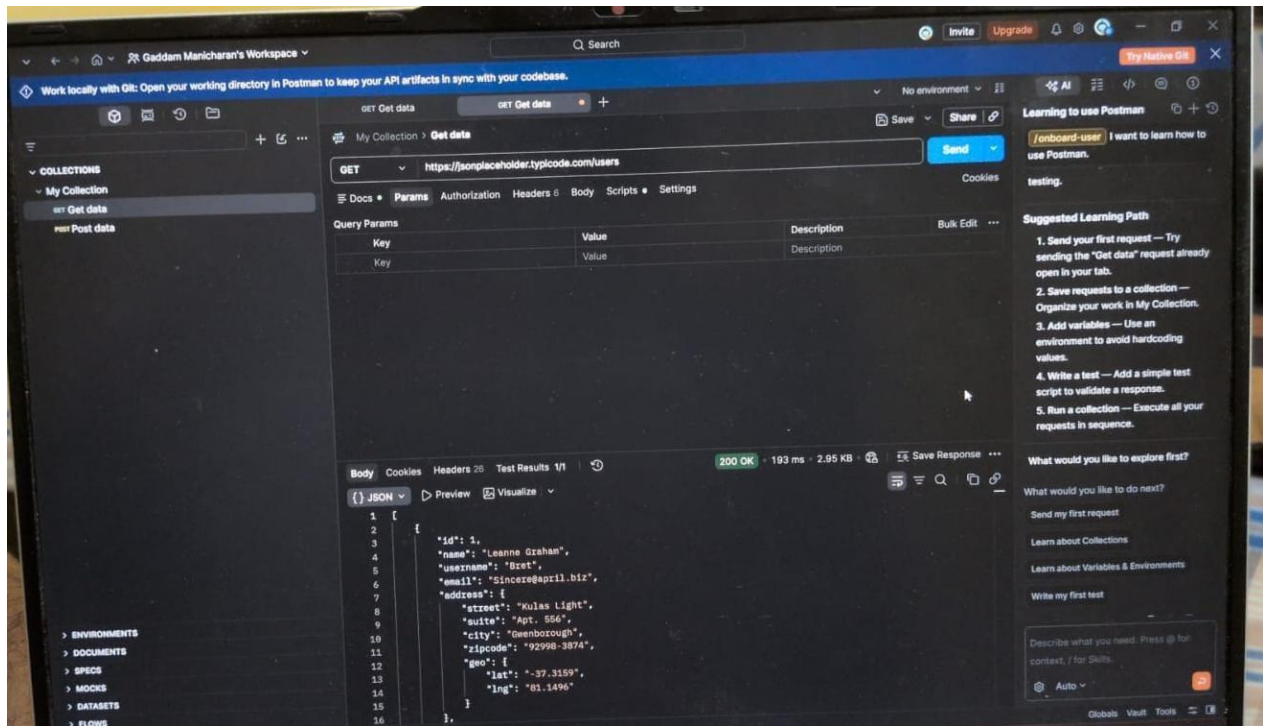
The POST /posts request returned HTTP 201 Created. The Postman test was updated to validate the expected 201 status. This demonstrates correct success-status validation for a resource-creation operation.

Recommended controls for real write endpoints:

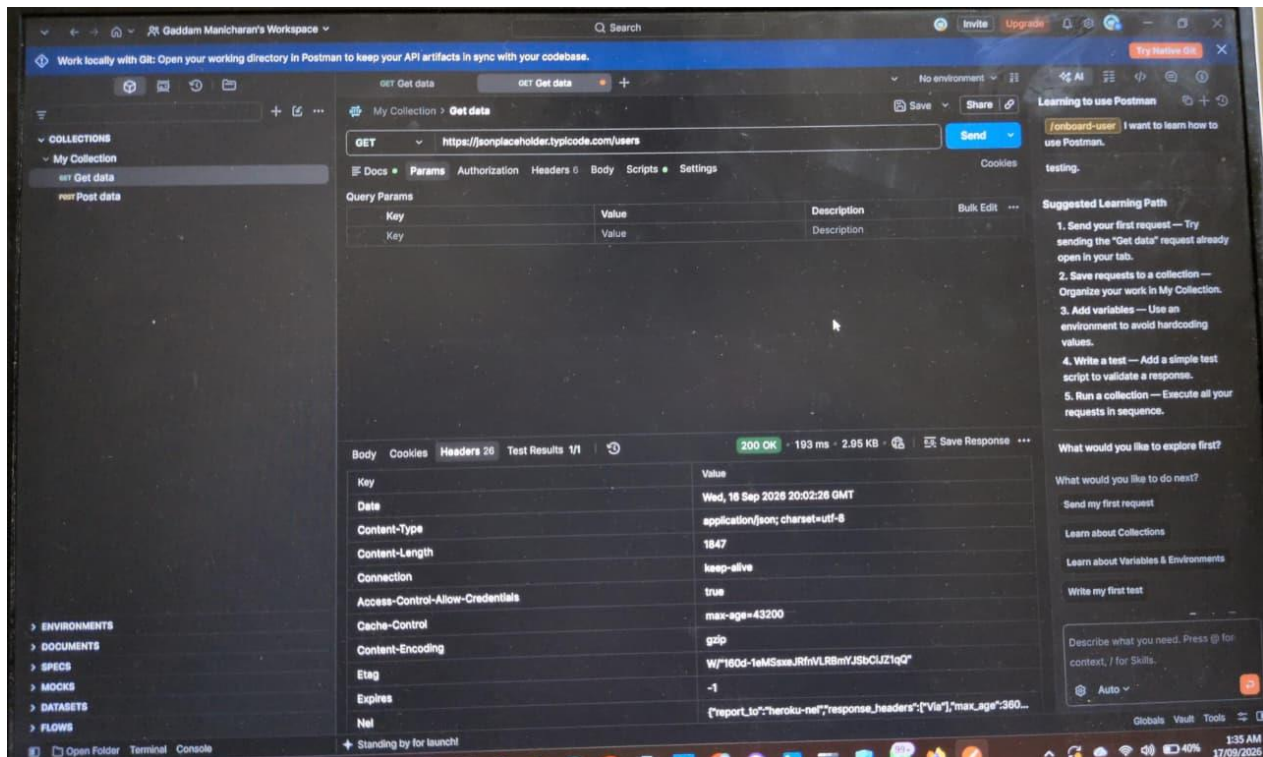
- Require authentication and authorization before creating resources.
- Validate and sanitize all input fields server-side.
- Apply request-size limits and content-type validation.
- Prevent mass assignment by allowing only explicitly approved fields.
- Log security-relevant creation events without storing secrets.

6. Evidence Appendix

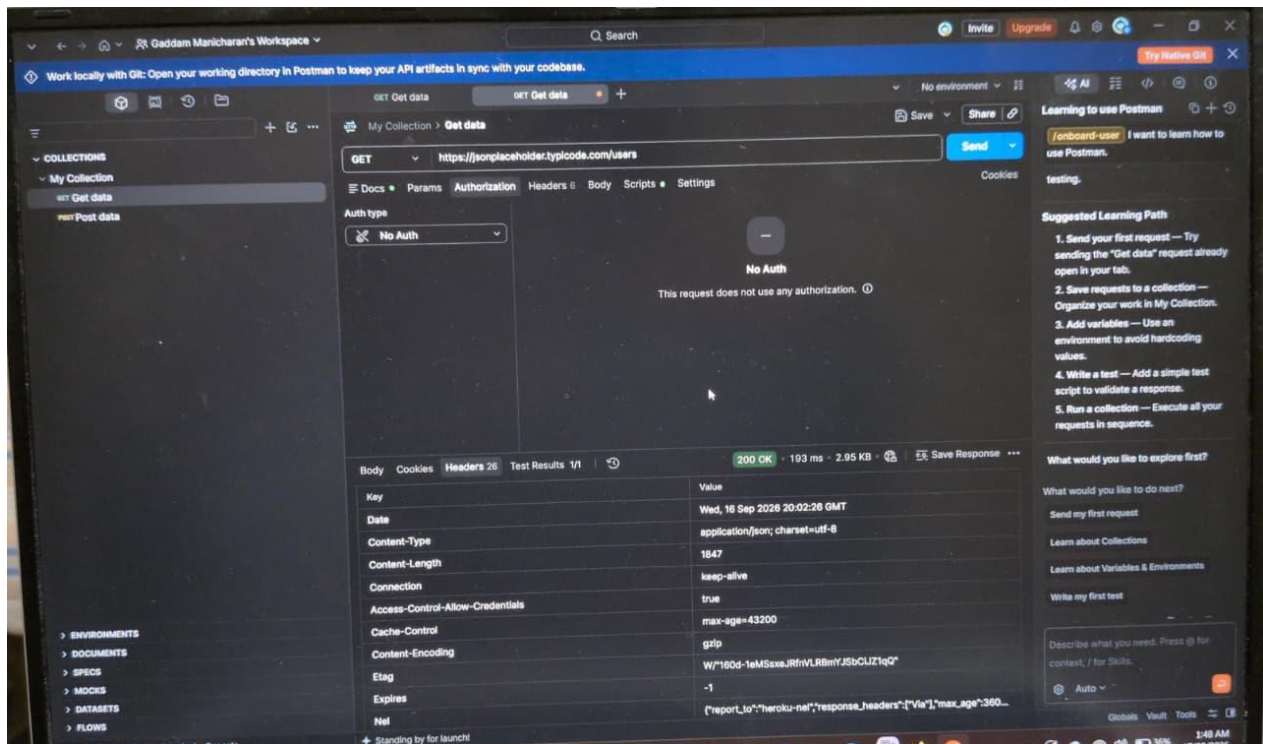
The following screenshots were captured during the Postman assessment. They are included as evidence of the observed requests, responses, headers, authentication configuration, and status codes.



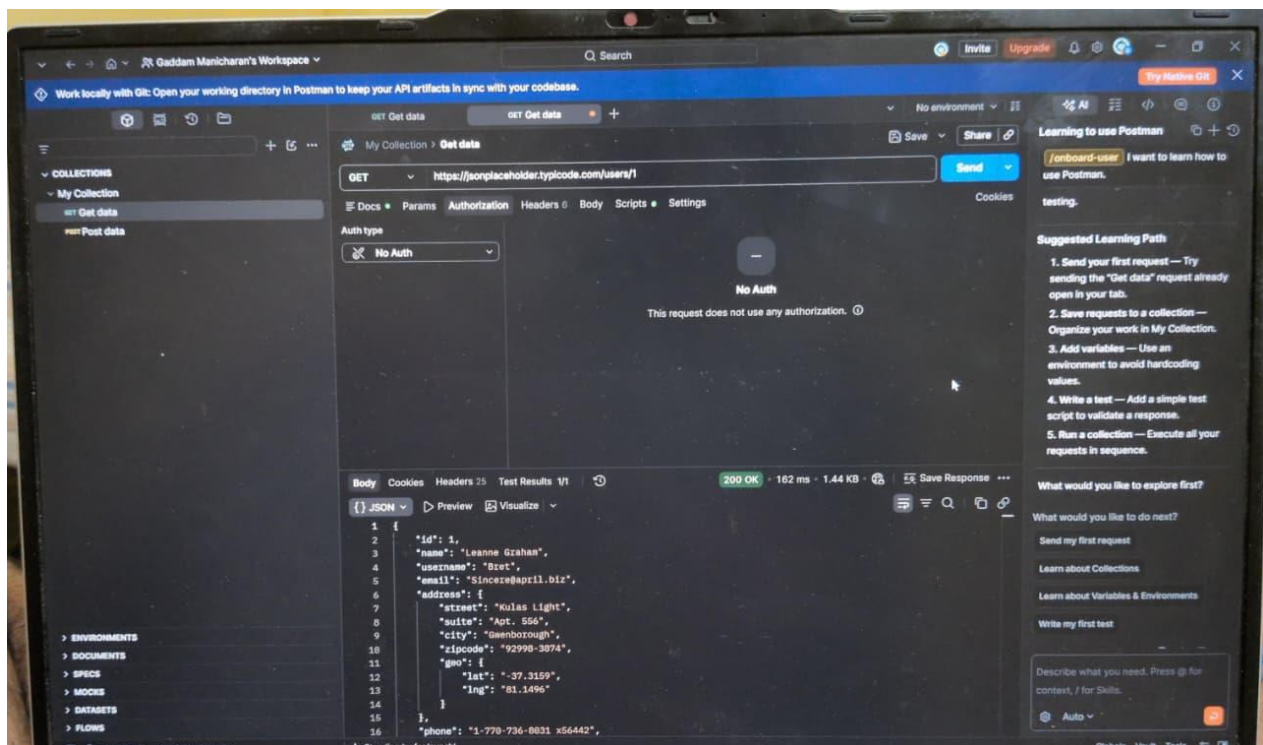
Evidence 01 — GET /users response (200 OK)



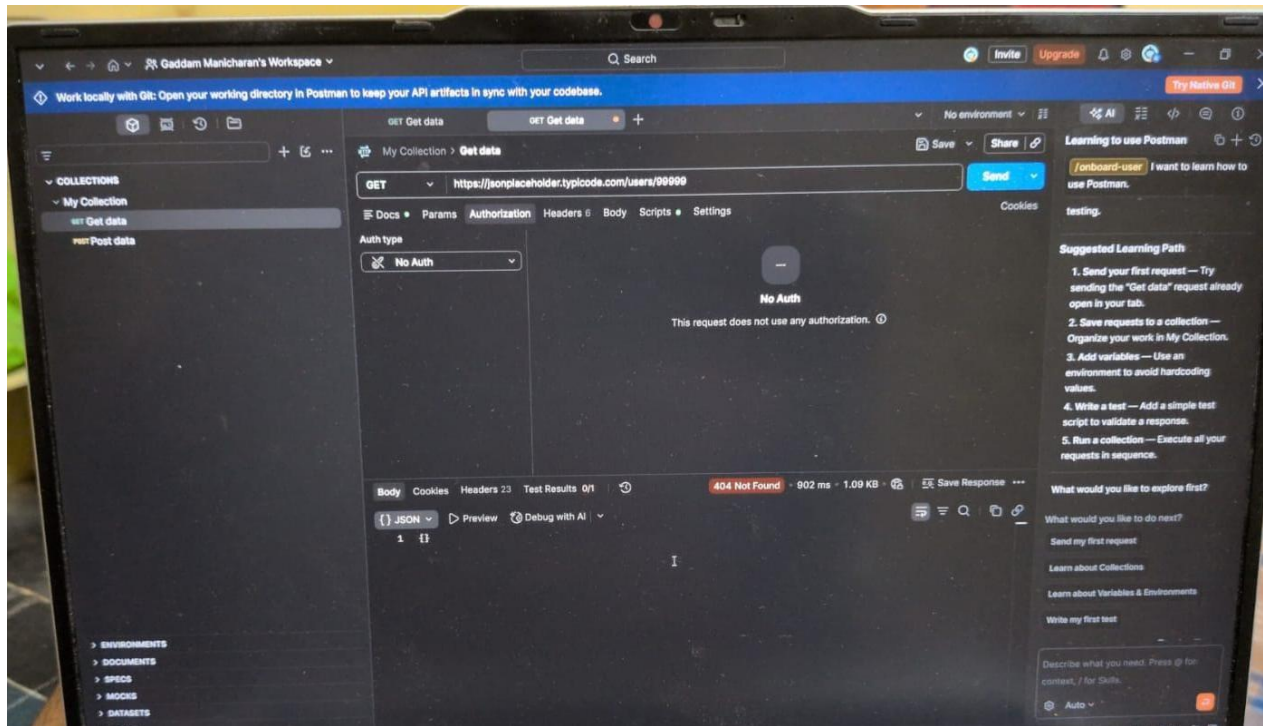
Evidence 02 — Response headers



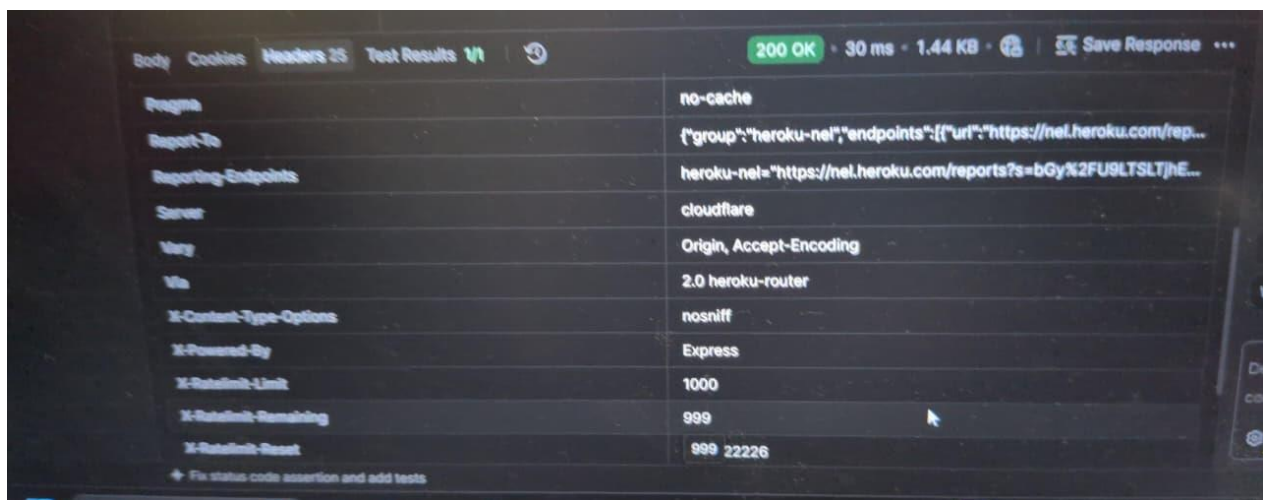
Evidence 03 — Authorization set to No Auth



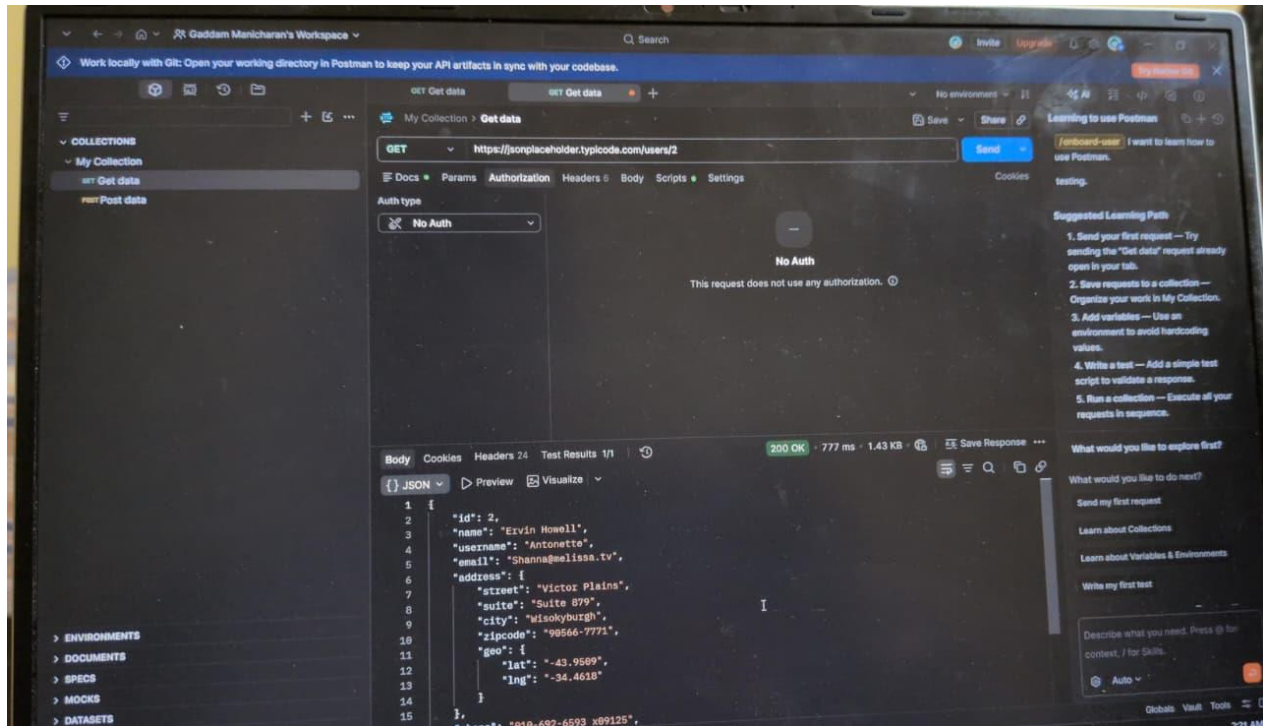
Evidence 04 — GET /users/1 user data response



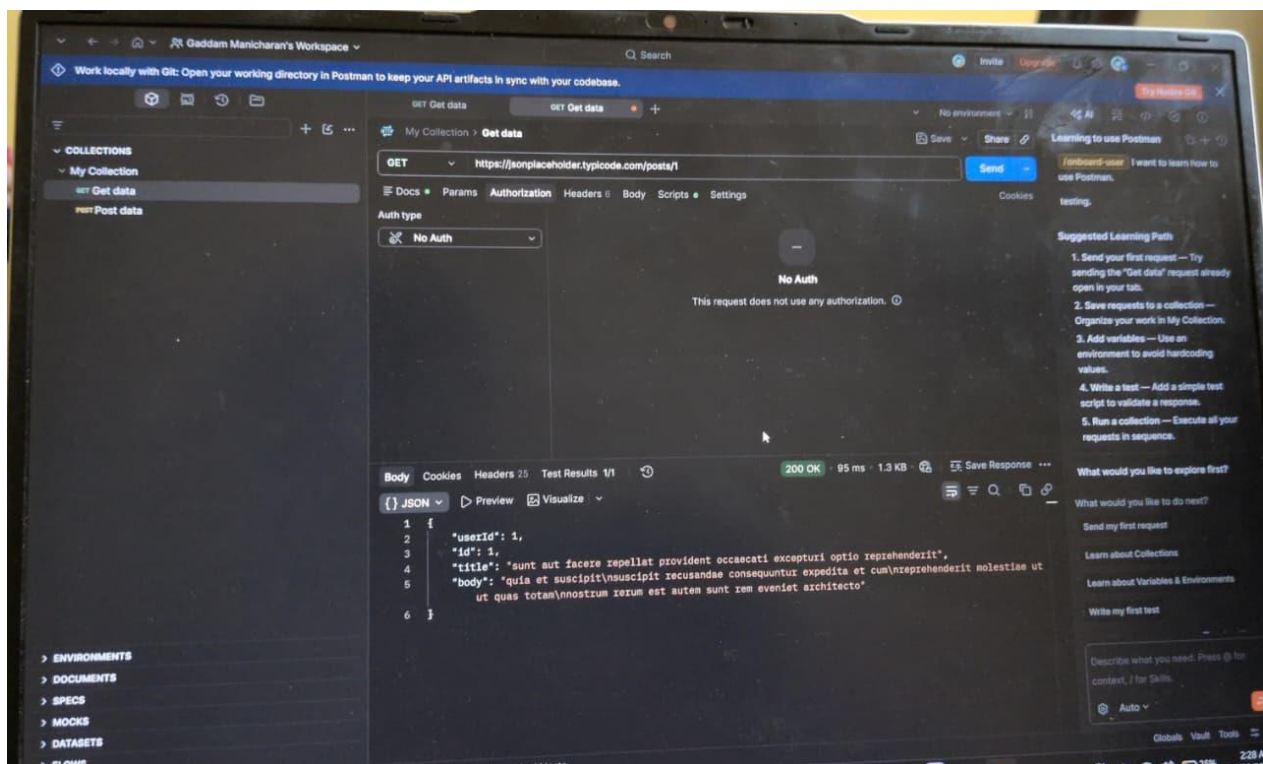
Evidence 05 — Invalid user ID returns 404 Not Found



Evidence 06 — Security headers and rate-limit response headers



Evidence 07 — Object access check for /users/1



Evidence 08 — POST /posts response

Body	Cookies	Headers 25	Test Results 1/1	200 OK • 908 ms • 1.31 KB • Save Response ...
Key	Value			
Date	Thu, 17 Sep 2026 19:12:05 GMT			
Content-Type	application/json; charset=utf-8			
Transfer-Encoding	chunked			
Connection	keep-alive			
Access-Control-Allow-Credentials	true			
Cache-Control	max-age=43200			
Etag	W/"124-yIKdLzqO5gfBrJFcdJ8Yq0LGnU"			
Expires	-1			
Nel	{report_to:"heroku-nel",response_headers:["Via"],"max_age":360...			
Pragma	no-cache			

Evidence 09 — Additional response headers

The screenshot shows the Postman interface with a POST request to `https://jsonplaceholder.typicode.com/posts`. The response is `201 Created` with the following headers:

Header	Value
Server	cloudflare
Vary	Origin, X-HTTP-Method-Override, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-RateLimit-Limit	1000
X-RateLimit-Remaining	999
X-RateLimit-Reset	1706711810
CF-Cache-Status	DYNAMIC
CF-Ray	a3ce2eb0694cd4db-NRT
Alt-Svc	h3="443"; ma=86400

On the right, a test script is shown with the following code:

```

1 pm.test("Status code is 200", function () {
2   pm.response.to.have
1   pm.test("Status code is 201", function () {
2   pm.response.to.have

```

The script is used to update the status code assertion from 200 to 201 to match the successful POST response.

Evidence 10 — Successful POST returning 201 Created

7. Risk Assessment Summary

The following risk assessment maps the observed API behavior to practical security considerations. Because JSONPlaceholder is a public educational/demo API and the assessment was non-destructive, these items are documented as risk observations or control considerations rather than confirmed production vulnerabilities.

Risk / Observation	Severity	Business Impact	Remediation
Public / unauthenticated endpoint exposure	Low	For this demo API, public access is expected. In a production SaaS API, unintended public access to private resources could expose data.	Require authentication for non-public resources and enforce access controls server-side.
Excessive data exposure potential	Medium	User responses contain multiple fields such as contact and address information. If unnecessary fields were exposed by a real application, privacy and data-minimization risks could result.	Return only fields required by the client; apply field-level response filtering and review sensitive-data exposure.
Object-level authorization (BOLA/IDOR) requires authenticated validation	Not confirmed	If a production API allowed one user to access another user's objects, unauthorized data disclosure could occur. The demo API does not provide the authenticated multi-user context needed to confirm this.	For authenticated APIs, enforce server-side authorization on every object and test cross-user access with authorized test accounts.
Rate-limiting control observed	Low	Rate-limit headers were present in the observed responses, indicating a control exists in the demo environment. Incorrect limits in a production API could still permit abuse or resource exhaustion.	Tune limits by endpoint, identity and risk; monitor abnormal request patterns and return consistent rate-limit responses.
Server / technology information disclosure	Low	Headers exposed implementation/proxy information such as X-Powered-By and Server/Via details. This can assist basic technology fingerprinting but does not demonstrate compromise.	Minimize unnecessary technology disclosure and apply an appropriate API security header policy.
Error handling for invalid resource	Low	An invalid user ID returned HTTP 404, which is predictable and	Standardize error responses and ensure internal details, stack

		does not expose an internal stack trace in the observed response.	traces and sensitive diagnostics are not returned.
Input validation	Not assessed	No intrusive validation or injection testing was performed. In production APIs, weak validation could affect data integrity and application security.	Validate type, format, length and allowed values on all client-supplied input, especially write operations.

Severity note: 'Low', 'Medium', and 'Not confirmed/Not assessed' reflect the limited evidence and scope of this educational assessment. They should not be interpreted as proof of exploitability in a production system.

8. Conclusion

The assessment successfully demonstrated a practical API security testing workflow using Postman. The tests covered authentication configuration, object-level access checks, data exposure review, error handling, security headers, rate limiting, and write-operation status validation.

The most important takeaway for a production SaaS environment is that successful API responses alone do not demonstrate secure authorization. Every protected object and sensitive field should be controlled by server-side authentication and authorization checks. The JSONPlaceholder results in this report are therefore treated as educational observations rather than confirmed production vulnerabilities.

9. Recommended Production Security Checklist

- Authentication required for all non-public resources.
- Server-side authorization for every object/resource.
- Least-privilege access to response fields.
- Consistent 401/403/404 error handling.
- Input validation and output schema enforcement.
- Rate limiting and abuse monitoring.
- Security headers appropriate to the API and deployment architecture.
- Centralized security logging and alerting.
- Automated API security tests in CI/CD.
- Regular penetration testing against authorized staging/production environments.

End of Report

Evidence note: The Postman screenshots already embedded in this report provide the evidence for the tested requests, responses, headers, authentication configuration, invalid-object handling, and POST operation.